

Project Title

HOSPITAL BED / OR SCHEDULING

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

P. Bhavan (2520090146)

M. Shiva (2520090135)

Team Number: 34

Section: 12

Under the guidance of

Ms. K. Chandusha

Asst. Prof., CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING /
COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The project report entitled “Hospital Bed / OR Scheduling” is a record of the bonafide work of P. Bhavan (2520090146) and M. Shiva (2520090135), submitted in partial fulfilment for the award of B.Tech in Computer Science and Information Technology at K L (Deemed to be) University. The work presented in this report is based on the supplied project abstract, presentation, implementation plan, and project README.

The project materials describe a DSA-3 zero-dependency optimization engine and its associated user interface for hospital bed and operating-room scheduling.

P. Bhavan (2520090146)

M. Shiva (2520090135)

K L (Deemed to be) University

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING /
COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



CERTIFICATE

This is to certify that the mini project based report entitled “Hospital Bed / OR Scheduling” is a bonafide work done and submitted by P. Bhavan (2520090146) and M. Shiva (2520090135) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science and Information Technology, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The successful completion of this project was made possible through the guidance and support of many people. We express our sincere gratitude to our project guide, Ms. K. Chandusha, Asst. Prof., CS&IT, for her valuable guidance, encouragement, and support throughout the development of the project.

We also thank the Department of Computer Science and Information Technology and the university for providing the academic environment and facilities required to complete this DSA-3 project. We are grateful to the teaching and non-teaching staff members who assisted us directly or indirectly.

Finally, we thank our parents, friends, and classmates for their cooperation and encouragement during the completion of this work

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)** Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P. Bhavan (2520090146)

M. Shiva (2520090135)

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	13
8.	Conclusion & Future Scope	14
9.	References	15

1.ABSTRACT

Hospitals face a complex scheduling problem involving patients, beds, operating rooms, surgeons, staff shifts, procedure durations, clinical priorities, and deadlines. The supplied project proposes a Hospital Bed / OR Scheduling Optimization Engine that allocates patients and procedures to rooms and time slots while minimizing makespan and idle turnaround time. The system combines data structures and algorithms from all six DSA-3 modules to model and solve different parts of the scheduling problem.

The implementation is designed as a zero-dependency Java engine in which core collections and algorithms are built from first principles without using java.util.* inside the engine. Graham's Longest Processing Time approximation is used for multi-OR makespan scheduling, bitmask dynamic programming models OR setup and sterilization ordering, and Dinic's maximum-flow/min-cut approach identifies capacity bottlenecks. Aho-Corasick and KMP support rapid emergency triage from clinical text, while bipartite matching and Vertex Cover address resource conflicts. Randomized and parallel techniques provide additional telemetry and sampling capabilities.

The project also specifies a responsive web dashboard with role-based login, an interactive multi-OR Gantt timeline, bed occupancy views, flow-network visualization, emergency triage, algorithm comparison, and dynamic simulation controls. Reporting features are intended to expose utilization, bottlenecks, and quality metrics and to support rapid rescheduling during emergencies, cancellations, and staff changes.

Team Members: P. Bhavan (2520090146), M. Shiva (2520090135).

2. INTRODUCTION

Hospital scheduling is a resource-allocation problem in which multiple activities must share limited beds, operating rooms, surgeons, staff, and time windows. The supplied presentation identifies three central operational challenges: idle resources, longer patient waits, and dynamic disruptions such as emergencies, cancellations, and staff absences. The project therefore treats scheduling as an optimization problem rather than a simple first-come-first-served allocation.

The proposed engine is designed to assign patients and procedures to available beds and operating rooms while considering bed types, OR availability, staff shifts, patient acuity, procedure duration, priorities, and deadlines. Its main objective is to improve utilization and throughput while reducing avoidable idle time and schedule makespan.

The architecture follows a modular pipeline. The input layer receives patient data, staff rosters, and room availability. A Java REST/CLI layer communicates with the core DSA engine, and the output layer presents schedules, allocations, dashboards, and alerts. The PPT architecture slide describes this separation so that individual layers can be updated or integrated with hospital information systems and electronic health records.

The presentation also emphasizes dynamic rescheduling. Emergency arrivals, procedure overruns, cancellations, and bed-sanitization delays can change the state of the hospital; the proposed simulation controls allow these events to be injected and the schedule to be re-optimized.

3. LITERATURE SURVEY

The supplied PPT states that healthcare scheduling research spans operations research, constraint programming, and machine learning. It highlights four findings relevant to this project: meta-heuristic approaches are effective for large dynamic instances; real-time rescheduling is important for emergency responsiveness; hybrid rule-based and optimization models are practical; and patient-priority weighting can improve clinical outcome metrics.

The present project follows a hybrid algorithmic approach. It uses exact algorithms where the problem size is manageable, approximation algorithms where the scheduling problem is computationally difficult, graph algorithms for capacity and matching, string algorithms for triage, and randomized/parallel algorithms for telemetry. This is consistent with the PPT's stated goal of combining operational efficiency with patient-centric scheduling.

Area	Approach represented in project	Purpose
Healthcare scheduling	Optimization / heuristic scheduling	Allocate limited beds, ORs, staff and time slots
Emergency triage	Aho-Corasick + KMP	Detect critical terms in clinical notes
OR turnaround	Bitmask DP / TSP formulation	Minimize setup and sterilization dead time
Capacity planning	Dinic / Max-Flow / Min-Cut	Identify structural bottlenecks
Resource conflict	Bipartite Matching + Vertex Cover	Resolve surgeon/OR and shared-resource conflicts
Large scheduling instances	Graham LPT approximation	Efficient multi-OR makespan scheduling
Exact benchmarking	Branch-and-Bound	Compare approximation against exact schedules

Note: the supplied project files do not provide a list of named external healthcare systems or academic papers. Therefore, this survey is limited to the research themes and algorithmic approaches explicitly described in the project PPT and implementation materials.

4. Problem Statement and Objectives

Problem Statement:

To develop a DSA-3 based optimization engine that dynamically schedules hospital beds and operating rooms while minimizing makespan and idle time and respecting patient priorities, procedure durations, staff availability, room constraints, and clinical deadlines.

Problem Description:

Manual scheduling can leave OR slots and beds idle while patients wait for suitable resources. The problem becomes more difficult when emergencies, cancellations, staff changes, or procedure delays require the schedule to be revised without creating resource conflicts.

Objectives:

1. Maximize utilization of hospital beds and operating rooms.
2. Minimize multi-OR makespan using an efficient approximation strategy.
3. Reduce inter-procedure setup and sterilization dead time.
4. Prioritize emergency and urgent patients while respecting operational constraints.
5. Identify capacity bottlenecks using network-flow and min-cut analysis.
6. Match surgeons and operating rooms according to resource requirements.
7. Support real-time emergency triage from clinical EHR text.
8. Provide transparent reports, dashboards, and quality metrics.
9. Demonstrate practical use of DSA-3 Modules 1–6 in one integrated system.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

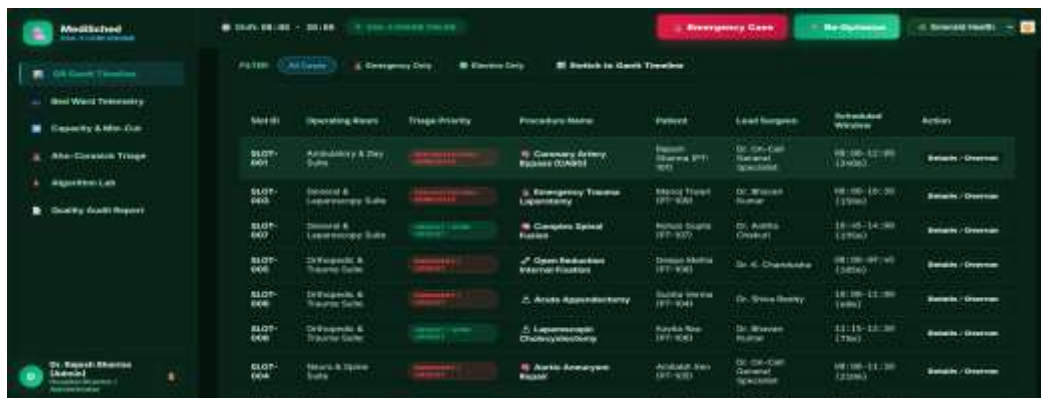
- Computer/laptop capable of running JDK 11 or later.
- At least 4 GB RAM recommended for development and dashboard execution.
- Dual-core or better processor recommended for running the Java engine and browser dashboard.
- Display and keyboard/mouse for interacting with the web dashboard.
- Network access is useful for external HIS/EHR integration, although the supplied engine is designed to run without third-party packages.

Software Requirements

- Java Development Kit (JDK) 11 or later.
- Java compiler (javac) and runtime (java) available on PATH.
- Web browser for the dashboard at <http://localhost:8080>.
- HTML5/CSS3/ES6-compatible browser environment.
- No Maven, Gradle, Docker, or third-party NPM packages are required according to the supplied project plan.

Development Environment

The implementation plan identifies a standalone Java backend with an embedded HTTP server and REST API, together with a modern web frontend. The backend can be launched through the terminal pipeline or through WebServerMain on port 8080.



Slot ID	Operating Room	Triage Priority	Procedure Name	Patient	Lead Surgeon	Scheduled Window	Action
SLOT-001	Amplification & Day Suite	Emergency (High)	Emergency Artery Repair (OR001)	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	08:00-12:00 (1200)	Bookable / Open
SLOT-005	General & Emergency Suite	Emergency (High)	Emergency Trauma Laparoscopy	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	08:00-12:00 (1200)	Bookable / Open
SLOT-007	General & Emergency Suite	Emergency (High)	Complex Spinal Fusion	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	13:00-17:00 (1200)	Bookable / Open
SLOT-009	Dr. Williams & Thomas Suite	Emergency (High)	Open Reduction Internal Fixation	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	08:00-12:00 (1200)	Bookable / Open
SLOT-010	Dr. Williams & Thomas Suite	Emergency (High)	Arthroscopic Meniscus Repair	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	13:00-17:00 (1200)	Bookable / Open
SLOT-011	Dr. Williams & Thomas Suite	Emergency (High)	Laparoscopic Cholecystectomy	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	13:00-17:00 (1200)	Bookable / Open
SLOT-013	Neuro & Spine Suite	Emergency (High)	Arthroscopic Meniscus Repair	Robert Brown (PT-001)	Dr. John Doe (Surgeon)	08:00-12:00 (1200)	Bookable / Open

6. IMPLEMENTATION

The project is organized as a zero-dependency Java core, a REST/HTTP integration layer, and a responsive frontend. The implementation plan maps the design directly to the six DSA-3 modules.

Module 1 – Custom Data Structures

The engine hand-builds CustomArrayList, CustomLinkedList, CustomHashMap, CustomMinHeap, CustomQueue, CustomStack, and CustomDisjointSetUnion. These structures support dynamic storage, indexing, priority handling, hashing, queues/stacks, and disjoint-set operations without importing `java.util.*` inside the core engine.

Module 2 – String Algorithms KMP, Z-Function, Rabin-Karp, Aho-Corasick, and Suffix Array/Kasai LCP are included. Aho-Corasick is the key emergency-triage component: it scans clinical text for multiple critical patterns such as cardiac arrest, massive hemorrhage, epidural hematoma, and septic shock and can promote the patient to a higher emergency priority.

Module 3 – Dynamic Programming

Wagner-Fischer edit distance supports typo-tolerant search, Needleman-Wunsch supports sequence alignment, Interval DP supports non-overlapping procedure allocation, and BitmaskTurnaroundDP models OR setup and sterilization ordering as a TSP-style optimization with $O(n^2 2^n)$ time.

Module 4 – Network Flow

Edmonds-Karp and Dinic maximum-flow algorithms model hospital capacity. Max-flow/min-cut analysis identifies bottleneck tiers, while bipartite matching supports surgeon-to-OR assignment. The frontend includes a flow visualization component for interpreting these bottlenecks.

Module 5 – NP-Hard Scheduling and Approximation

Graham's LPT algorithm is used for parallel-machine makespan minimization. The implementation plan gives the bound $4/3 - 1/(3m)$ times OPT for m identical machines. Exact Branch-and-Bound is used for smaller workloads to obtain an optimal benchmark, and Vertex Cover 2-Approximation is used for shared-resource conflict resolution.

Module 6 – Randomized and Parallel Algorithms

Randomized QuickSort provides expected $O(n \log n)$ sorting, Universal Hashing supports lookup, Reservoir Sampling supports uniform sampling from a stream of emergency arrivals, and Blelloch Parallel Prefix Sum is used for cumulative workload telemetry with $O(n)$ work and $O(\log n)$ span.

System Architecture

The architecture shown in the PPT separates the system into Input Layer, Core Engine, and Output & Reporting. The implementation plan further specifies a Java HTTP REST server with authentication, scheduling, flow, bed/room, triage, simulation, and reporting handlers. The frontend contains login/role selection, Gantt scheduling, bed grids, flow visualization, triage, algorithm comparison, and turnaround optimization views.

RESULTS & DISCUSSION

The supplied project files define the expected functional outputs and verification plan but do not contain measured benchmark tables or actual application-run screenshots. Therefore, this section reports the outputs that the supplied design and README specify, without inventing numerical performance results.

Expected Application Outputs

- An optimized multi-OR schedule displayed as an interactive Gantt timeline.
- Bed occupancy status for ICU, PACU, Surgical, and General wards.
- Capacity-flow visualization with the min-cut bottleneck highlighted.
- Emergency EHR triage results with detected keywords and priority information.
- Comparison between Graham LPT approximation and exact Branch-and-Bound scheduling.
- Dynamic rescheduling after Code Blue arrival, trauma surge, OR overrun, or bed-sanitization delay.
- Analytics and reports covering utilization, bottlenecks, and quality metrics.

Verification and Testing

The implementation plan specifies unit tests for each custom data structure and algorithmic tests for KMP, Z-Function, Rabin-Karp, Aho-Corasick, edit distance, Bitmask TSP, Edmonds-Karp, Dinic, min-cut, LPT versus Branch-and-Bound, randomized QuickSort, and Reservoir Sampling. A full-system integration test is also specified for realistic hospital simulation cycles.

Manual Verification Flow

1. Compile the Java sources using javac and place compiled classes in the bin directory.
2. Start WebServerMain on port 8080.
3. Open <http://localhost:8080> in a browser.
4. Log in using the demonstration credentials described in the supplied README.
5. Toggle dark/light mode and inspect the Gantt, bed, flow, triage, and comparison views.
6. Trigger emergency and overrun simulation events and verify that the schedule is re-optimized without conflicts.

7. CONCLUSION & FUTURE SCOPE

Conclusion

The Hospital Bed / OR Scheduling project demonstrates how Data Structures and Algorithms can be applied to a practical healthcare operations problem. The supplied design combines custom data structures, string matching, dynamic programming, network flow, approximation algorithms, randomized algorithms, and parallel prefix processing in one scheduling engine.

The project addresses the stated objectives by providing mechanisms for multi-OR scheduling, bed management, emergency prioritization, turnaround optimization, bottleneck identification, resource matching, dynamic simulation, and reporting. The modular architecture also supports integration with hospital information systems and EHR sources through REST APIs.

Because the supplied materials do not include empirical benchmark results, this report does not claim specific numerical improvements in makespan, utilization, or patient waiting time. Such values should be added after executing the system on a defined dataset and recording the resulting metrics.

Future Scope

- Connect the engine to live hospital information systems and EHR feeds.
- Use historical hospital data to calibrate procedure durations, demand patterns, and priority weights.
- Extend optimization to multi-day and multi-department scheduling.
- Add predictive models for emergency arrivals, bed demand, and procedure overruns.
- Add stronger role-based access control and audit logging for production deployment.
- Evaluate algorithm performance using larger real or synthetic datasets and publish quantitative benchmark results.
- Extend the simulation layer with additional disruption scenarios and recovery policies.
- Explore hybrid optimization approaches combining rule-based constraints, meta-heuristics, and exact methods.

8. REFERENCES

1. Supplied project abstract: “DSA 3 ABSTRACT” – Hospital Bed / OR Scheduling, Team 34, Section 12.
2. Supplied project presentation: “Hospital Bed & OR Scheduling” – methodology, literature survey, architecture, and project overview slides.
3. Supplied implementation plan: “Implementation Plan: Hospital Bed & Operating Room (OR) Scheduling Optimization Engine”.
4. Supplied project README: “Real-Time Hospital Bed & Operating Room (OR) Scheduling Optimization Engine”.
5. KL (Deemed to be) University, Data Structures and Algorithms–3 (25CS2103E) project documentation template supplied with the task.
6. Algorithm and module descriptions specified in the supplied implementation plan, including DSA-3 Modules 1–6.

Note: The template requests external URLs for similar projects, but the supplied files do not provide specific external URLs. No external web sources were introduced into the technical claims of this report.